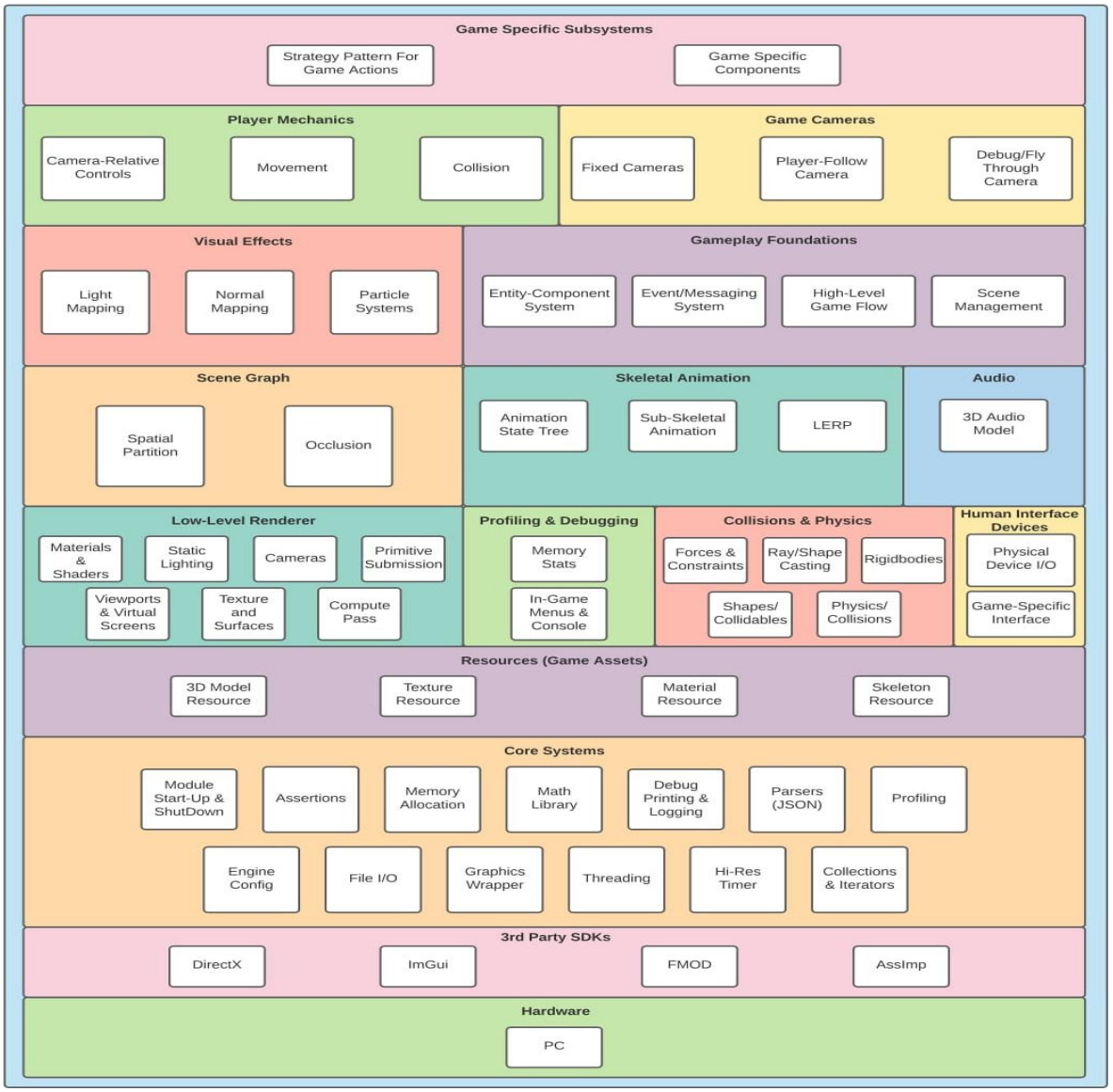
Technical Design Document

Game Engine Features

- Game Engine Architecture
- Renderer
- Object & Material Loader
- HLSL Shaders
- Compute Rendering
- Real-Time Physics
- Animations
- Entity-Component System
- Scene Creation and Management
- Navigation Mesh Creation
- Multithreading
- Audio via FMOD
- Input
- Dynamic Window
- Event Handling
- UI via ImGui
- File I/O
- Data Structures & Design Pattern

Game Engine Architecture



Renderer

DirectX 12 renderer that abstracts rendering out of the game entirely through a RenderComponent. Following the Microsoft Mini Engine examples and source code, we have included an abstracted rendering context with graphics and compute implementations, various buffer implementations, pipeline state and other DirectX object abstractions, and support for lower DirectX feature tiers (ie. support for machines that can only support DirectX 11).

Fully Supported Object & Material Loader

Using the AssImp (Asset Importer) library, the engine can load models using a wide variety of file extensions, such as .obj, .fbx, .dae, and can load in the associated materials (.mtl) and associated textures. Objects can be loaded asynchronously and at run-time using a deferred loading scheme.

HLSL Shaders

We've written shaders for the colour render pass, animated objects, and our particle system using compute shaders. We support compiling shaders at run-time and using offline pre-compiled shaders.

Compute Rendering

Support for compute passes for features such as particles. Asynchronous buffer reading and writing, atomic GPU counters, and compute shaders are supported, allowing for engine systems to be scaled using GPU threads. Allows for buffers used in compute pass to be transferred to render pass to render compute results.

Real-Time Physics

Rigidbody component for kinematic movements with other inclusions such as inertia tensors and orientation. Collision detection for spheres, boxes, and complex colliders (utilizing the GJK and EPA algorithms). Collision response for simple objects.

Animations

Using the assimp library to load in bone data as well as the other model data to create animations.

Entity-Component System

Allows for easy adding and removing of components from GameObjects. GameObjects initialize added components and manage their lifetime. Components can reference parent GameObject and thereby reference other attached components.

Scene Creation and Management

Scenes can be implemented from the abstract Scene interface, allowing for them to be attached to the game interface. Scenes can initialize GameObjects and any custom objects

and can apply logic to any object. Scenes do not need to manage the lifetime of GameObjects and do not need to update GameObjects, as they are managed by a hidden scene objects manager.

Navigation Mesh Creation

Custom navigation meshes can be created for single-layer, flat scenes with predetermined obstacles. Navigation meshes are created using a multistep process, starting with creating a binary grid from the received scene data, then by using a marching squares algorithm to generate contour lines, reducing those contour lines via the Ramer-Douglas-Peucker point reduction algorithm, using an open source API for Delauney triangulation, culling obscured triangles, generating navigation data for pathfinding, and finally generating a mesh for the navigation mesh.

Multithreading

Multithreading is used in two contexts - our custom job system, and in multiple manager style static-objects such as the TextureManager, SceneObjects manager, and others. Our custom job system allows for binding functions to the job system, whereby a worker thread will execute the job in a separate thread. This system uses fences to allow for synchronization with the main thread.

Audio via FMOD

Utilizing FMOD for 3D-positional audio. Supports multiple channels allowing for easy management of multiple sound files.

Input

Input manager which accesses hardware inputs through the mouse and keyboard.

Dynamic Window Sizing & Support for any Aspect Ratio

Support for dynamic window sizing during application run-time, which dynamically adjusts the aspect ratio of the rendering engine.

Event Handling

Support for events such as application events, window events, and input events. Events will filter through the system layers in the engine, being received by any listening system, and being removed when “handled” by an appropriate system.

UI via ImGui

Utilizing ImGui to create adjustable and customizable widgets, including displaying engine profiling data, and supports ability to create UI elements for game use.

File I/O

Basic file input and output, with more customized support for JSON file reading and writing for game and engine data.

Data Structures & Design Patterns

Custom built data structures such as graphs and ring buffers, and design patterns such as the observer pattern or the strategy pattern.